

Course Outline: Async Queue Processing (Offloading)

Title: Async Queue Processing: Offloading Work with Queues **Skill:** Skill 47: Using queues and tasks to delegate **Learnpack:** + Procesamiento asíncrono con colas (Offloading) **File:** s47-w16d47-procesamiento-asincrono-con-colas-offloa **Prerequisite:** Skill 46 (Queue Management: The Waiting Room Pattern) **Target Audience:** Beginner developers who understand queue fundamentals and now need to implement async processing with the producer-consumer pattern **Total Lessons:** 10 **Format:** Mixed (10% hands-on = 1 lesson)

Course Description

You know what a queue is. Now learn how to use one. This course covers the full cycle of async queue processing: how a request becomes a job, how a worker consumes and processes it, what message brokers add (durability, distribution, acknowledgment), what happens when things go wrong (failure handling, dead letter queues, retry), how to tell users their work is in progress, and the production design principles that keep queue-based systems reliable. The hands-on challenge ties it all together with a Python producer-consumer implementation.

Course Philosophy

This course emphasizes:

- The flow from request to job to result — understanding the lifecycle
 - Reliability as a design requirement, not an afterthought
 - Building the mental model before using higher-level tools (RabbitMQ in the next learnpack)
-

Course Statistics Summary

Section	Lessons
00 - Welcome	1
01 - The Async Queue Pattern	3
02 - Reliability and Production	4
03 - Assessment	1
04 - Outro	1
Total	10

Learning Objectives:

- Understand what "offloading" means in the context of web applications
- Recognize the async queue processing lifecycle: request → enqueue → process
- Preview the course structure

Content Outline:

- What "offloading" means: moving work out of the request handler and into a background system
- The three-step lifecycle: receive the request → enqueue the task → process the task asynchronously
- Why offloading matters: users get fast responses; heavy work happens in the background without blocking
- The roles in this system: the producer (request handler), the queue (buffer), and the worker (processor)
- Connection to what students already know from Skill 46: the queue is the same FIFO structure; this course teaches how to use it in a production async system
- Course structure: Section 01 builds the conceptual model (how the pattern works, workers, brokers); Section 02 covers production reliability (failures, DLQ, progress communication, design principles); Section 02 ends with the hands-on challenge

Transitions: This lesson sets up the problem. Section 01 walks through the complete pattern.

Key Principles:

- The queue is passive — it holds work; the worker is active — it executes work
- Offloading does not make work go away; it makes work visible and deferrable
- The pattern scales horizontally: add more workers to increase throughput

Examples:

- Without offloading: `POST /generate-report` → wait 45 seconds → return PDF (user waits)
 - With offloading: `POST /generate-report` → enqueue job → return `{job_id: "j-99"}` → user polls `GET /jobs/j-99` → after 45 seconds: `{status: "done", url: "/reports/r-99.pdf"}`
-

01.0 How Async Queue Processing Works

Learning Objectives:

- Trace the complete request-to-result lifecycle in an async queue system
- Identify where the producer, queue, and consumer operate in a web application
- Explain why the request handler must return immediately after enqueueing

Content Outline:

- The lifecycle step by step:
 1. **Receive:** HTTP request arrives at the handler
 2. **Enqueue:** handler creates a job description and adds it to the queue
 3. **Return:** handler immediately returns a response (job ID, status URL, or just a 200)

- 4. **Dequeue:** worker picks up the job from the queue
- 5. **Execute:** worker processes the job
- 6. **Result:** worker stores the result or triggers a notification
- Why the handler must return immediately: the handler is in the request-response cycle; keeping it open for a slow task blocks the HTTP worker thread and degrades performance
- The decoupling principle: the producer doesn't know which worker will process its job, or when — it only knows the job is queued
- The tradeoff: the user no longer gets an immediate answer — they get a reference to a pending job
- Job representation: a job description contains everything the worker needs to execute independently: input data, task type, any necessary context (user ID, configuration, etc.)
- Section preview: 01.1 covers how workers consume jobs; 01.2 covers message brokers

Transitions: You understand the lifecycle. 01.1 shows how the worker side works.

Key Principles:

- The handler's responsibility ends at "enqueue and return" — it does not wait for results
- The job description must be self-contained — the worker has no access to the original request context
- Any state the worker needs must be serialized into the job

Examples:

- Enqueue call: `job_queue.put({"task": "send_email", "to": "user@example.com", "template": "welcome", "user_id": 42})`
- Handler returns: `{"status": "queued", "job_id": "j-abc123"}`
- Worker picks up: `pops {"task": "send_email", ...}` → sends the email

01.1 The Worker Pattern

Learning Objectives:

- Define a worker as a process that continuously pulls jobs from a queue and executes them
- Describe the work loop: dequeue → process → acknowledge → repeat
- Explain how multiple workers enable horizontal scaling

Content Outline:

- What a worker is: a long-running process (or thread) that watches the queue and processes jobs one by one
- The work loop:
 1. Dequeue: take the next job from the queue
 2. Process: execute the task described by the job
 3. Acknowledge: signal to the queue that the job was completed (so the queue won't redeliver it)
 4. Repeat: go back to step 1

- What happens if the worker crashes mid-job: without acknowledgment, the queue keeps the job as "in-flight" and can redeliver it to another worker (this is why idempotency — from Skill 45 — is critical)
- The `get()` operation blocks: if the queue is empty, the worker waits (does not spin in a busy loop); this is the `queue.Queue.get()` behavior in Python
- **Horizontal scaling with multiple workers:**
 - Start with one worker; when throughput is insufficient, start a second; both pull from the same queue
 - The queue ensures each job is processed exactly once (at-least-once with acknowledgment)
 - Each worker is independent — one crashing doesn't affect the others
- Worker shutdown: workers typically watch for a sentinel value (None, a special job type) or a signal to stop cleanly
- Section preview: 01.2 covers message brokers — the production-grade systems that implement the queue + worker model with durability and distribution

Transitions: You understand the worker side of the pattern. 01.2 shows what message brokers add to this picture.

Key Principles:

- A worker is stateless — it processes jobs independently; all necessary state comes from the job description
- Multiple workers = higher throughput; the queue serializes coordination
- The acknowledge step is what makes the system fault-tolerant: unacknowledged jobs can be reassigned

Examples:

```
# Simple worker loop
while True:
    job = task_queue.get() # blocks until a job is available
    if job is None:
        break # shutdown signal
    process(job)           # execute the task
    task_queue.task_done() # acknowledge completion
```

01.2 Message Brokers

Learning Objectives:

- Define a message broker as a system that implements queue + worker coordination with production-grade guarantees
- Name the key guarantees brokers add: durability, acknowledgment, distribution, backpressure
- Identify common brokers (Redis Queue, Celery, AWS SQS) and their positioning

Content Outline:

- What an in-memory queue lacks for production use:
 - No durability: if the process crashes, queued jobs are lost
 - No distribution: only accessible within the same process
 - No acknowledgment guarantees: no built-in re-delivery of failed jobs
- What a message broker adds:
 - **Durability**: jobs are persisted (to disk or a distributed store) — they survive crashes
 - **Acknowledgment**: the broker tracks which jobs are "in-flight"; unacknowledged jobs are redelivered after a timeout
 - **Distribution**: producers and consumers can run on different machines; the broker is the intermediary
 - **Backpressure**: when the queue grows too large, the broker can slow down producers or alert operators
- Common message brokers:
 - **Redis Queue / Celery + Redis**: Redis stores job data in lists (O(1) LPUSH/RPOP); fast, widely used in Python; optional persistence
 - **AWS SQS**: fully managed, serverless, infinite durability; integrates with Lambda; adds some latency vs Redis
 - The next learnpack covers **RabbitMQ** in depth: exchanges, routing, AMQP protocol
- The broker as the canonical "queue" from Skill 46: the queue data structure you studied is what these brokers implement at scale with added guarantees
- What students will learn in Skill 47 (RabbitMQ): producer → exchange → queue → consumer architecture; ACK/NACK; multiple workers; Docker setup

Transitions: You now understand the full pattern and the tools. Section 02 covers what makes this pattern reliable in production.

Key Principles:

- A message broker is a queue + durability + distribution — the FIFO principle still applies
- Acknowledgment is the single most important guarantee for correctness: it prevents message loss
- Choose the broker based on your needs: Redis/Celery for simple Python apps; SQS for managed AWS infrastructure; RabbitMQ for complex routing

Examples:

- Redis Queue: `from rq import Queue; q = Queue(connection=Redis()); q.enqueue(send_email, user_id=42)`
- Celery: `@app.task def process_order(order_id): ...; process_order.delay(99)`
- SQS: `sqs.send_message(QueueUrl='...', MessageBody=json.dumps(job))`

02.0 Reliability and Production

Learning Objectives:

- Name the reliability requirements for a production async queue system

- Understand why "working" and "reliable" are different bars for queue-based systems
- Preview the topics in Section 02

Content Outline:

- The reliability gap: a working queue system processes jobs; a reliable one processes them correctly even when things go wrong
- The three production reliability requirements:
 1. **Failure handling**: know when jobs fail and have a plan for re-running them
 2. **Dead letter queue (DLQ)**: jobs that fail repeatedly are moved to a holding area for investigation
 3. **Progress communication**: users who submitted jobs need to know their status
- Plus the design principles that prevent problems: 4. **Idempotency**: every job must be safe to run multiple times (covered conceptually in Skill 45 — now applied practically) 5. **Resilience design**: lightweight messages, horizontal scaling, task prioritization 6. **Observability**: status tracking and logging for every job
- Section preview: 02.1 covers failure handling and DLQ; 02.2 covers communicating progress to the frontend; 02.3 is the hands-on challenge

Transitions: This overview sets up the reliability dimension. 02.1 starts with what happens when jobs fail.

Key Principles:

- A queue that loses jobs, silently fails jobs, or duplicates work is worse than not having a queue
- Reliability is a design decision made upfront, not an optimization added later
- The five reliability dimensions (failure, DLQ, progress, idempotency, observability) are interconnected

Examples:

- Without failure handling: a job that fails once is simply lost — nobody knows
- Without DLQ: a poison pill job (one that always fails) retries forever, blocking the queue
- Without progress communication: the user who submitted the job sees a spinner indefinitely

02.1 Failure Handling and DLQ

Learning Objectives:

- Implement retry logic for failed jobs with exponential backoff
- Define a dead letter queue (DLQ) and explain when jobs are moved there
- Explain why idempotency is mandatory for any job that may be retried

Content Outline:

- **What is job failure?**
 - A job fails when the worker throws an exception while processing it
 - Causes: transient (network timeout, service down) vs permanent (bad data, code bug)

- The worker catches the exception, decides whether to retry, and either re-enqueues or DLQs the job
- **Retry logic:**
 - Automatic retry on transient failures — the job is re-enqueued after a delay
 - Exponential backoff: wait 1s → 2s → 4s → 8s between retries to avoid hammering a failing service
 - Maximum retries: after N attempts (e.g., 3-5), stop retrying
 - What to log: job ID, attempt number, error message, next retry time
- **Dead Letter Queue (DLQ):**
 - A special queue that receives jobs that have exceeded their maximum retries
 - Purpose: preserve failed jobs for investigation and manual re-processing
 - DLQ jobs are not lost — they can be inspected, fixed, and re-queued manually
 - Implementation: after `max_retries` is exceeded, move the job to a `dead_letter_queue` instead of discarding it
- **Idempotency — why it's mandatory:**
 - Retried jobs run more than once — if the job is not idempotent, side effects (emails sent twice, payments charged twice, rows inserted twice) occur
 - Every job in a retry-enabled system must be designed to be safe to run multiple times
 - Implementation patterns: check before act (look up by idempotency key), upsert instead of insert, record processed job IDs
- **Gestión de Reintentos y Fallos — the curriculum requirement:** "What happens if the processing fails due to a network error? Does it retry automatically? Is it discarded? Is it saved in a 'failure table' for manual review?" — the answer is: retry with backoff, then DLQ

Transitions: Failure handling protects the system. 02.2 covers how to tell users what's happening with their jobs.

Key Principles:

- Never discard a failed job without logging it and moving it to a DLQ
- Exponential backoff prevents the retry pattern from overwhelming a failing dependency
- Idempotency and retry logic must be designed together — you cannot add retries to a non-idempotent job without breaking correctness

Examples:

```
def process_job_with_retry(job, max_retries=3):
    for attempt in range(max_retries):
        try:
            process(job)
            return # success
        except Exception as e:
            wait_seconds = 2 ** attempt
            log({"job_id": job["id"], "attempt": attempt+1, "error": str(e),
                "retry_in": wait_seconds})
            if attempt < max_retries - 1:
                time.sleep(wait_seconds)
    # max retries exceeded
```

```
dead_letter_queue.put(job)
log({"job_id": job["id"], "status": "dead_lettered"})
```

02.2 Communicating Progress

Learning Objectives:

- Implement a job status store that tracks pending → processing → completed/failed states
- Choose between polling and push (webhooks, WebSockets) for communicating job status to the frontend
- Explain the tradeoffs between polling and push for different use cases

Content Outline:

- **The problem:** the user submitted a job and is waiting. How do they know when it's done?
- **Job status model:**
 - Create a status record when the job is enqueued: `{job_id, status: "pending", created_at}`
 - Worker updates to "processing" when it starts: `{status: "processing", started_at}`
 - Worker updates to "completed" or "failed" when done: `{status: "completed", finished_at, result_url}` or `{status: "failed", error}`
 - Status is stored in a database, Redis, or any persistent store the API can read
- **Approach 1: Polling**
 - The frontend periodically calls `GET /jobs/{job_id}` to check status
 - Simple to implement; works everywhere
 - Tradeoff: latency = polling interval; can create unnecessary load if many clients poll frequently
 - Good for: tasks that take minutes, low-frequency checks, simple implementations
 - Polling interval: start at 2-5 seconds; back off if still pending after 30 seconds
- **Approach 2: Push (WebSocket / SSE)**
 - The backend pushes a status update to the client when the job completes
 - No polling overhead; near-real-time notifications
 - More complex to implement (requires persistent connection or webhook endpoint)
 - Good for: tasks where timing is critical, high-frequency status updates, real-time dashboards
- **"Comunicación del progreso al Frontend" from the curriculum:** this is exactly what status stores + polling or push implement
- Connecting to observability: the same job status record that the API exposes to users is also the observability data for operators

Transitions: Status communication closes the user-facing feedback loop. 02.3 is the hands-on challenge that puts the whole pattern together.

Key Principles:

- A job without a visible status is invisible to the user — always create a status record at enqueue time
- The status record is the contract between the backend and the frontend: it's how the async system communicates completion
- Polling is simpler and more resilient; push is faster but adds infrastructure complexity

Examples:

```
# Polling endpoint
@app.get("/jobs/{job_id}")
def get_job_status(job_id: str):
    job = job_store.get(job_id)
    if not job:
        raise HTTPException(404)
    return {"job_id": job_id, "status": job["status"], "result":
job.get("result")}
```

Frontend polling:

```
async function pollJobStatus(jobId) {
    while (true) {
        const res = await fetch(`/jobs/${jobId}`);
        const data = await res.json();
        if (data.status === 'completed') return data.result;
        if (data.status === 'failed') throw new Error(data.error);
        await sleep(3000); // poll every 3 seconds
    }
}
```

02.3 Implement a Producer-Consumer

Challenge Prompt: Write a Python producer-consumer system using `queue.Queue` and threading. The producer enqueues 5 tasks with IDs and values. A single worker dequeues each task, simulates processing with a short sleep, and prints a structured JSON log for each step: "enqueued", "processing", and "done" with the computed result (`value × 2`).

Solution Code:

```
import queue
import threading
import json
import time

task_queue = queue.Queue()
results = []
```

```

def worker():
    while True:
        task = task_queue.get()
        if task is None:
            break
        print(json.dumps({"status": "processing", "task_id": task["id"]}))
        time.sleep(0.1)
        result = {"task_id": task["id"], "status": "done", "result":
task["value"] * 2}
        results.append(result)
        print(json.dumps(result))
        task_queue.task_done()

worker_thread = threading.Thread(target=worker)
worker_thread.start()

for i in range(5):
    task = {"id": i, "value": i + 1}
    print(json.dumps({"status": "enqueued", "task_id": i}))
    task_queue.put(task)

task_queue.join()
task_queue.put(None)
worker_thread.join()

print(f"\nProcessed {len(results)} tasks")

```

Challenge Code:

```

import queue
import threading
import json
import time

task_queue = queue.Queue()
results = []

def worker():
    # Continuously dequeue tasks until a None sentinel is received.
    # For each task: print {"status": "processing", "task_id": ...}
    # Simulate work with time.sleep(0.1)
    # Compute result = task["value"] * 2
    # Append {"task_id": ..., "status": "done", "result": ...} to results
    # Print the result dict as JSON
    # Call task_queue.task_done()
    # your code here
    pass

# Start the worker thread
# your code here

```

```
# Enqueue 5 tasks: id=0..4, value=id+1
# Print {"status": "enqueued", "task_id": id} for each
# your code here

# Wait for all tasks to finish, send shutdown signal (None), join thread
# Print total processed count
# your code here
```

03.0 Assessment 🧠

Learning Objectives:

- Apply async queue processing concepts to new scenarios
- Identify reliability gaps in described systems
- Predict behavior of queue-based systems under failure conditions

Question Format: Scenario-based (multiple-choice)

Questions:

1. A request handler calls `job_queue.put(job)` and immediately returns `{"status": "queued"}`. The user receives this response in 50ms. The job itself takes 2 minutes. This pattern is called:
 - A) Synchronous processing
 - B) Offloading (async queue processing) ✓
 - C) Caching
 - D) Parallel processing
2. A worker dequeues a job and starts processing. Halfway through, the worker crashes. What happens next in a system with proper acknowledgment?
 - A) The job is permanently lost
 - B) The job is automatically marked as "failed" in the database
 - C) The broker redelivers the unacknowledged job to another worker ✓
 - D) The producer re-enqueues the job automatically
3. A payment processing job runs successfully but the worker crashes before sending the acknowledgment. As a result, the broker redelivers the job and it runs again. If the payment job is NOT idempotent, what happens?
 - A) Nothing — the second run is a no-op
 - B) The user is charged twice ✓
 - C) The second run fails because the payment already exists
 - D) The job goes to the DLQ automatically
4. A job fails 3 times due to a database timeout. After the 3rd attempt, the job is moved to the dead letter queue. What should happen next?

- A) The job is permanently deleted
 - B) The system automatically retries the job every hour indefinitely
 - C) A developer investigates the DLQ, fixes the issue, and manually re-queues the job ✓
 - D) The system sends the job back to the original queue with higher priority
5. You want to show a user the progress of their report generation job. The job takes 30 seconds. What is the simplest approach?
- A) WebSocket connection from the browser to the worker
 - B) Have the handler wait for the job to complete before returning
 - C) Store job status in a database; let the frontend poll `GET /jobs/{id}` every 3 seconds ✓
 - D) Send the result directly to the user's email
6. What is the primary benefit of using exponential backoff in retry logic?
- A) It ensures the job eventually succeeds
 - B) It prevents retries from overwhelming a service that is already struggling ✓
 - C) It prioritizes failed jobs over new jobs
 - D) It deduplicates jobs that were enqueued multiple times
7. You add 10 workers to your queue-based email system. What is the expected effect?
- A) The queue fills up 10x faster
 - B) Each worker processes a different queue
 - C) Throughput increases approximately 10x (each worker processes its own set of jobs) ✓
 - D) The queue automatically splits into 10 sub-queues
8. Your job description contains the entire user profile object (500KB). What production design principle does this violate?
- A) Idempotency
 - B) Durability
 - C) Lightweight messages ✓
 - D) Acknowledgment

Evaluation Criteria:

- Correctly identifies the lifecycle step and role of each component
- Understands what acknowledgment prevents
- Applies idempotency reasoning to production scenarios
- Recognizes reliability antipatterns

Score Interpretation:

- 8/8: Ready to implement production queue-based systems
 - 6–7: Review any missed lessons before moving on
 - 4–5: Re-read Sections 01 and 02
 - Below 4: Restart from Section 01
-

04.0 Async Processing Patterns

Content Outline:

- Course recap: what students accomplished
 - Traced the full async queue lifecycle: receive → enqueue → return → dequeue → process → result
 - Learned the worker loop: dequeue → process → acknowledge → repeat
 - Understood message brokers and what they add (durability, distribution, backpressure)
 - Built the reliability picture: failure handling + DLQ + progress communication + idempotency + observability
 - Wrote a working Python producer-consumer with structured logging
- Connection to bigger picture: this learnpack covered async queue processing conceptually and with Python's in-memory queue. The next learnpack (RabbitMQ) shows how these exact patterns look when implemented with a production message broker: exchanges, routing, persistent queues, and multiple worker processes
- Next steps: Skill 47 (RabbitMQ) — the AMQP protocol, producer-exchange-queue-consumer architecture, Docker setup, ACK/NACK in pika
- Resources for further exploration:
 - Python [threading](#) and [queue.Queue](#) documentation
 - Celery documentation — the most widely used Python task queue framework
 - AWS SQS documentation — managed queue service with built-in DLQ support
 - "Designing Distributed Systems" — patterns for worker processes and task queues
- Encouragement: the pattern you learned here — producer → queue → worker → result — is the same pattern used by every major async system in production. The tools change (RabbitMQ, SQS, Celery, Kafka), but the lifecycle, the failure handling, the idempotency requirement, and the progress communication need remain constant. Build on this foundation and every message queue system you encounter will feel familiar.

Note: This is conclusion only — no theory, exercises, or assessment.